

科目：データベース演習

第3回：論理設計とパフォーマンス, DB作成

講師：鈴木源吾

前回の復習

1. 解説：システム開発とデータベース設計
 - システム開発の工程
 - 設計工程とデータベース
2. 例題：Google Colabを使ったデータベース作成・検索
3. 解説：データベースの設計
4. 演習：大学データベースの作成

今回のトピック

1. 解説：データベースのパフォーマンス
 - パフォーマンスに影響する要因：テーブルの非正規化
 - パフォーマンスに影響する要因：インデックス設計
2. 解説：データの作成
3. 例題：ゲームデータベースのDB作成
4. 演習：シラバスデータベースのDB作成

topic 1

解説：データベースのパフォーマンス

論理設計と物理設計

論理設計・物理設計でやるべきことの基本を演習し，データベースの作成までを行う

1. テーブル・カラムの決定
2. データ型の決定
3. 参照整合性の決定（主キー・外部キーの設定）
4. テーブルの作成
5. インデックスの作成
6. データの作成

今回は，主に5を実施する

システムの非機能要件とデータベース設計

- データベース設計では、パフォーマンス（例：応答性能）や可用性（例：稼働時間）などの非機能要件の考慮も必要となる
- 今回は、その中でも基本的なパフォーマンスに影響する設計を学ぶ
- パフォーマンスを意識する設計段階は、主に物理設計（下図は第4回資料より）

	データモデル	特徴	やること（例）	アウトプット（例）
概念設計	ERモデル	DBMSに依存せず実世界を理想的に記述	ER図の作成	ER図
論理設計	リレーショナルモデル	DBMSも合わせた論理的構造を記述	テーブル・カラム・データ型の決定	テーブル設計書（概念スキーマ）
物理設計	リレーショナルモデル	実際に物理的に格納・管理するための記述	インデックス付与, 分散・冗長化, 格納領域・パラメータ決定	テーブル作成SQL・パラメータ設定, 等（内部スキーマ）

注：3層スキーマのうち外部スキーマはアプリケーションに依存するため、データベース設計のアウトプットに含めなかった

データベースのパフォーマンスに影響する要因

今回、パフォーマンスに影響する要因として、以下の2つについて説明する

1. テーブルの非正規化
2. インデックス設計

この他に、例えば、ストレージの冗長構成（RAID）やファイルの物理配置も実用的には非常に重要な話だが、今回は割愛する。興味のある人は、参考書を参照すること

正規化の振り返り

リレーショナルデータモデルの正規化は、様々な更新時異状を発生させないために必要であると、データベースの基礎で学んだ（第7回）

- 更新時異状とは、例えば、一つのデータを削除したときに、同時に他の事実も失われてしまうような状況だった
- それを防ぐために、リレーションを分解し、一つのもの（実体）は一つのリレーションだけに入るようにした（正規化）

正規化されたテーブルの例

- 下図は典型的な正規化されたテーブルである
- 会社・社員・部署という実体が、きちんと分解されている
- それぞれのテーブルの主キーに下線をひいた
(社員的主キーは、会社コードと社員IDの複合キー)

会社

<u>会社コード</u>	会社名
C0001	A 商事
C0002	B 化学
C0003	C 建設

社員

<u>会社コード</u>	<u>社員ID</u>	社員名	年齢	部署コード
C0001	000A	加藤	40	D01
C0001	000B	藤本	32	D02
C0001	001F	三島	50	D03
C0002	000A	斉藤	47	D03
C0002	009F	田島	25	D01
C0002	010A	渋谷	33	D04

部署

<u>部署コード</u>	部署名
D01	開発
D02	人事
D03	営業
D04	総務

正規化されたテーブルに対する問合せ

正規化されたテーブルから情報を得ようとすれば、複数のテーブルを結合する操作が増えることになる

例) 田島さんの勤めている会社と部署を知りたい

```
SELECT 会社.会社名, 部署.部署名  
FROM 社員, 会社, 部署  
WHERE 社員.社員名 = '田島' AND  
      社員.会社コード = 会社.会社コード AND  
      社員.部署コード = 部署.部署コード;
```

→ 結合の増加 → 性能劣化の可能性あり

- 結合は非常にコストの高い操作. 多いテーブル数の結合は、レコード数が増える
と性能が劣化する

非正規化による解決

性能の劣化を避けるために、正規化を諦めることが実用上はある。

これを**非正規化**と呼ぶ。下のテーブルの場合、前の問合せは以下ですむことになる

```
SELECT 会社名,部署名 FROM 社員 WHERE 社員名 = '田島';
```

社員

<u>会社コード</u>	会社名	<u>社員ID</u>	社員名	年齢	部署コード	部署名
C0001	A 商事	000A	加藤	40	D01	開発
C0001	A 商事	000B	藤本	32	D02	人事
C0001	A 商事	001F	三島	50	D03	営業
C0002	B 化学	000A	斉藤	47	D03	営業
C0002	B 化学	009F	田島	25	D01	開発
C0002	B 化学	010A	渋谷	33	D04	総務

非正規化による情報の損失

しかし、p.10のテーブル 社員には、p.8のテーブル 会社に含まれていた「会社コード C0003である会社の会社名がC建設」であるという情報が損失している。

- データベースの基礎 第7回で学んだ、更新時異状に相当
- テーブル 社員に、C建設の社員のレコードがないことが原因
- かといって、(C0003, C建設, null, ..., null,) という情報を追加すれば、社員IDが主キー（の一部）である制約に矛盾してしまう

→ 非正規化は、情報を損失なく、整合性を保って維持するには、望ましくない

注：非正規化した社員と合わせて、会社テーブルも保持しておけばこの問題は発生しない

正規化・非正規化と更新の関係

一方、正規化・非正規化と更新との関係はどうなるか？

→ 正規化：1つのレコード更新でよい、非正規化：多くのレコードの更新要

例：A商事の会社名がE物産に変更

正規化

```
UPDATE 会社 SET 会社名 = 'E物産' WHERE 会社名 = 'A商事';
```

非正規化

```
UPDATE 社員 SET 会社名 = 'E物産' WHERE 会社名 = 'A商事';
```

正規化・非正規化の判断は？

エンジニア・理論家によって意見のわかれるところ

- ある人：正規化は必須で、いかなる理由があっても非正規化すべきでない
- 別の人：最初から非正規化を論理設計の中に織り込むべき

データ整合性とパフォーマンスのトレードオフだが・・・

データベースの大家、クリス・デイトによると（参考書より）

「非正規化」はあくまでも最後の手段

非正規化の誘惑にかられても、他の手段によってパフォーマンス向上が図れないかを最後の最後まで検討すべき

インデックスによる性能向上

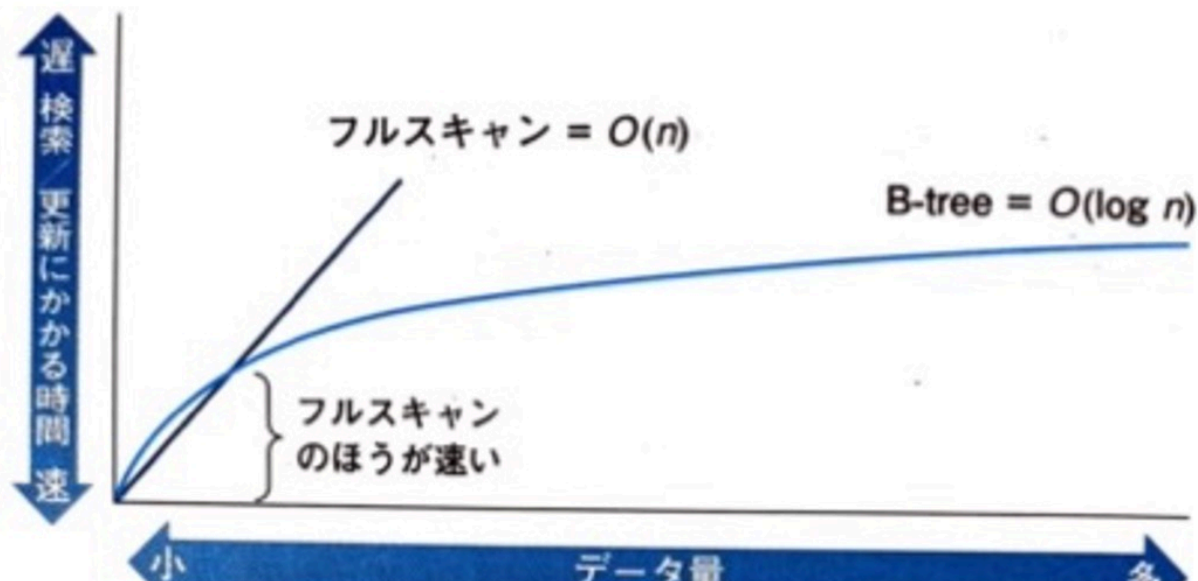
インデックスはデータベースの基礎第9回で学んだ (B+木). インデックスはSQLのパフォーマンス向上のための非常にポピュラーな手段. 理由を以下にあげる

- アプリケーションのコードに影響を与えない
 - データベース側にインデックスを作成すればよいだけ
- テーブルのデータに影響を与えない
- それでいて性能向上の効果が大きい
 - n (フルスキャン) から $\log n$ (ツリーの高さ) のオーダーに向上

B木（B+木）インデックスの設計方針

指針1：大規模なテーブルに対して作成する

- データ量が小さい場合は，フルスキャンのほうが速いケースがある（下図）
 - オプティマイザ（最適化器）がインデックスを使うかどうかを判断する
- データが主記憶にすべて乗るくらい小さければ，フルスキャンで十分速い
 - 目安としてレコード数1万以下だと，インデックスの効果なし（参考書）



B木（B+木）インデックスの設計方針

指針2：データの値が分類され、その種類が「適度に」多いカラムに作成する

- 例：性別のような2値（男性・女性）のカラムにインデックスを作成しない
 - 値の種類が少なければ、インデックスのツリーは小さく効果が少ない
- 例：備考のような、自由な文字列情報が入り、すべてのカラムで値が異なるようなカラムにB木インデックスを作成しない
 - インデックスの容量が大きくなり、性能が劣化することがある
 - 後述の指針3により、インデックスが利用されない場合も多い
 - 全文検索（テキスト検索）機能の活用を検討すべき
- 特定のキー値を指定したときに、全レコード数の5%程度に絞り込めれば効果あり
 - 受付日のようなカラムは、365日のうち1日を指定する問合せを考えれば、0.3%に絞り込めるので効果ありと言える

B木（B+木） インデックスの設計方針

指針3：SQL文でWHERE句の選択条件・または結合条件に使用されそうな列に作成する

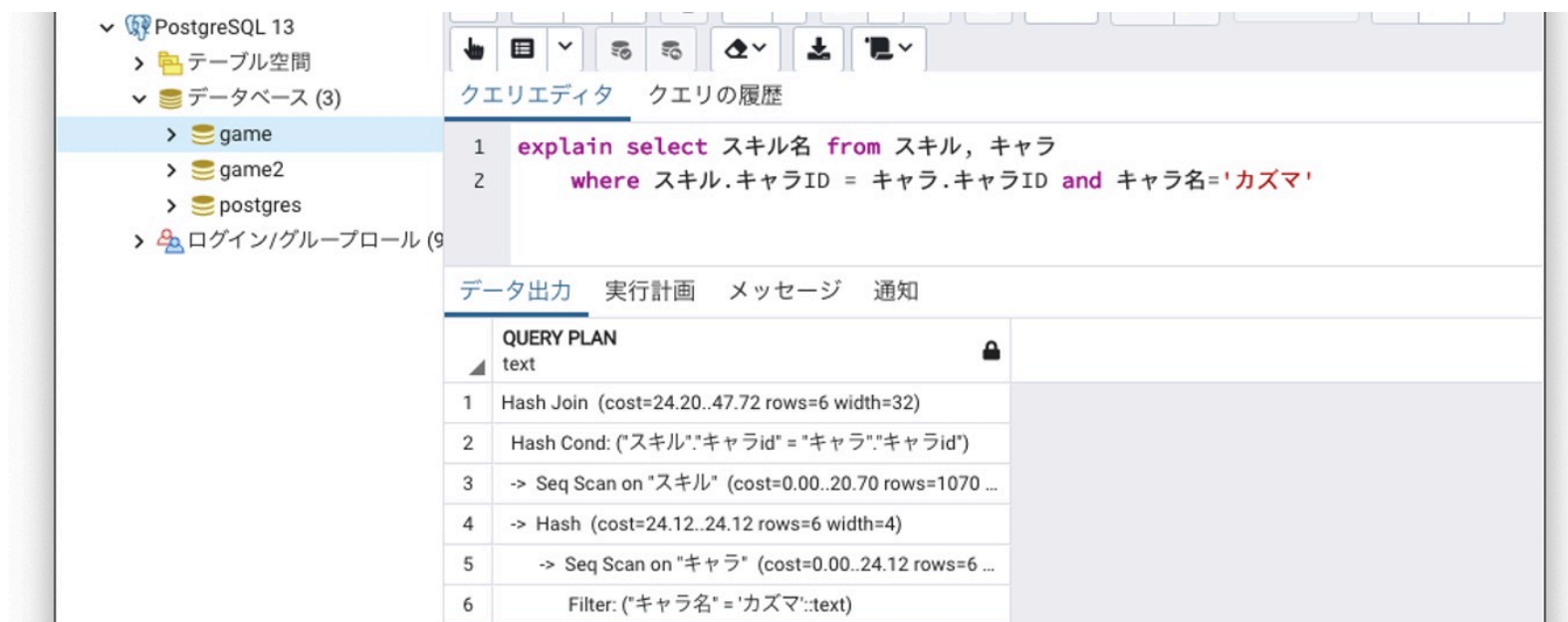
- データベースの基礎でも学んだが、インデックスが使用されるには、問合せの中で条件値が指定されなければいけないから
- 注意：選択条件でも、以下の場合にはインデックスが使われにくい
 - 例：LIKE句の中間一致検索（LIKE '%検索文字列%'）
 - 例：ORを用いたWHERE句
 - 例：カラムに演算や関数を適用している場合（例：YEAR(受付日) = 2025）

補足：主キーにはほとんどのDBMSで自動的にインデックスが作成されるため、別途インデックスを追加する必要なし

（PostgreSQLの場合、pg_indexesテーブルで確認できる）

インデックス使用を確認するためには？

PostgreSQLでは、EXPLAINというコマンドが用意されており、EXPLAIN SQL文、と指定することで、実行プラン（SQL文が変換されたデータアクセスの手続き、データベースの基礎第10回を参照）を出すことができる（以下、実行例、Seq Scan→Sequential Scan（順スキャン）なのでインデックスは使われていない）



The screenshot shows the PostgreSQL 13 interface. On the left, the database structure is visible, including 'game', 'game2', and 'postgres' databases. The main window displays a query in the 'クエリエディタ' (Query Editor) tab:

```
1 explain select スキル名 from スキル, キャラ
2 where スキル.キャラID = キャラ.キャラID and キャラ名='カズマ'
```

Below the query editor, the 'データ出力' (Data Output) tab is active, showing the 'QUERY PLAN' (Query Plan) for the executed query. The plan is as follows:

Step	Operation	Cost	Rows	Width
1	Hash Join	(cost=24.20..47.72)	rows=6	width=32
2	Hash Cond: ("スキル".キャラid = "キャラ".キャラid)			
3	-> Seq Scan on "スキル"	(cost=0.00..20.70)	rows=1070	
4	-> Hash	(cost=24.12..24.12)	rows=6	width=4
5	-> Seq Scan on "キャラ"	(cost=0.00..24.12)	rows=6	
6	Filter: ("キャラ名" = 'カズマ':text)			

topic 2

解説：データの作成

データの作成

INSERT文を使って行を作成することができる(データベースの基礎 第5回)

例) ゲームデータベースのメンバーのレコードを1つ作成する

```
INSERT INTO メンバー (HP) VALUES (800)
```

注意

- serial型のメンバーIDは指定しない. 自動的に番号が生成される
- メンバーIDの指定をしないので, ()によるカラムの指定が必須
 - 参考: () でカラムを指定しない場合, 全カラムの値をVALUES以降で指定する

インデックスの作成

CREATE INDEX文を使ってインデックスを生成することができる

基本となる書式

```
CREATE INDEX [ インデックスの名前 ] ON [ ONLY ] テーブル名  
    ( カラム名 [, ...] )
```

- カラムを複数指定したインデックスの作成も可能（カラム複数の値に対してインデックスができる。複数の値を条件に指定した検索が高速になる）
- インデックスが作成されたことは、pg_indexesテーブルを参照すればわかる

実行例：テーブル スキルのカラム スキル名にskill_name_idxという名前のインデックスを作成する

```
CREATE INDEX skill_name_idx ON スキル (スキル名)
```

CSVファイルのエクスポート・インポート

PostgreSQLでは、COPYコマンドを使って、テーブルの中身をCSVファイルに保存したり（エクスポート、COPY TO）、CSVファイルのデータをテーブルにインポート（COPY FROM）したりできる

- DBサーバプロセスを実行するOSユーザ（postgres）のファイルの読み・書き権限要、文字コード、Windows・Macによる違いなど、設定には注意が必要
 - 対処例：全員が読み書き実行可能なフォルダを作って、CSVファイルを置く
- 興味がある人は、以下のマニュアルなどの情報を参考に試してほしい
 - COPY <https://www.postgresql.jp/document/15/html/sql-copy.html>
 - PostgreSQL コマンド COPY <https://dev.classmethod.jp/articles/postgresql-copy-command/>

topic 3

演習：インデックスの作成と効果の確認

第3回演習のノートブックに従って実施すること

課題1

第2回・第3回の演習のノートブックを実施し，WebClassに共有リンクで提出すること